

DEMO 09 • NEURAL INTEROP

The Bridge Between Neural Toolchains

A vector-in, vector-out interop demo that reads a Nengo-style stream, transforms it, and emits an output vector with a confidence — proving Jneopallium can sit between neural systems.

ADVISORY Recommends only — no autonomous writes

AUTHOR Rakovskyi Dmytro • VERSION Full-run demo 09 • DATE 2026 • REPOSITORY github.com/rakovpublic/jneopallium

Real neuromorphic and simulation stacks rarely live alone — they need something to read their output, reason about it, and hand a result to the next stage. This demo casts Jneopallium in that role. It consumes a deterministic vector stream in the shape a Nengo model would produce, runs it through a small transform-and-decide network, and emits an output vector with an explicit confidence. No Python and no Nengo install are required to run it.

What it is

Demo 09 models interoperability with a Nengo-style neural toolchain. Its safety ceiling is **ADVISORY**: it emits an output vector and a confidence as a recommendation to a downstream system, not a control action. The network is four layers (sized 3·3·2·2), enough to extract temporal features and produce a classified or control-advisory output.

How it is wired

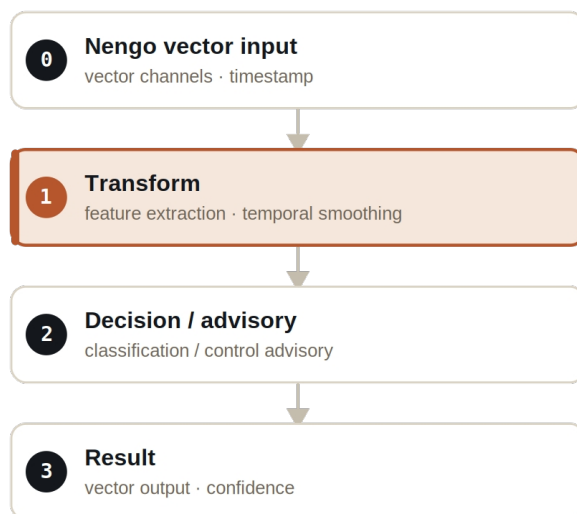


Figure Four layers. A transform layer smooths the vector stream before a decision layer produces an advisory output.

Signals are typed: `NengoVectorSignal` carries the incoming channels and timestamp, `TemporalFeatureSignal` carries the smoothed features, and `NengoOutputSignal` carries the output vector and its confidence.

What it does

- It reads a deterministic two-channel vector stream — a mock sine/cosine source standing in for a real Nengo model.
- It extracts temporal features and smooths them, then produces an output vector for each input frame.
- Every output carries a confidence value, so a downstream consumer knows how much to trust each frame.

NOTE

The mock source is the point: the demo proves the interop contract — vectors in, vectors-with-confidence out — without requiring a Python Nengo runtime to be installed.

The evidence

VERIFICATION EVIDENCE

Input frames	deterministic two-channel stream
Output rows	75 vector outputs

Confidence	emitted per output (> 0)
Output mode	ADVISORY
Verdict	PASS

How to run it

1 Build

Build the worker and the demo model JAR.

```
mvn -q -pl worker -am package
```

2 Run via the framework entry point

Entry loads the model JAR and the demo context through the real local path.

```
java -cp demo-model.jar \  
  com.rakovpublic.jneuropallium.worker.application.Entry \  
  local file:///path/to/demo-model.jar \  
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \  
  context.json
```

3 Inspect

Read the output vectors and their confidences.

```
ls target/jneopallium-fullrun-demos/demo-09-nengo-interop/
```

Why ADVISORY

An interop bridge should inform the next stage, not commandeer it; the receiving system owns whatever happens next. The advisory ceiling keeps Jneopallium a well-behaved middle stage. To connect a real toolchain you replace the mock vector source with a live Nengo process feed while preserving the typed signal contract and the advisory ceiling — the output vector and confidence arrive the same way, and the downstream system decides what to do with them.